
CSCI 566 - Optimizing Mathematical Reasoning Performance of Small-Scale Language Models via CoT-SFT-GRPO

Neural Overthinkers

Abstract

This project examines how small language models can strengthen multi-step reasoning through lightweight training. We apply Supervised Fine-Tuning (SFT), Group Relative Policy Optimization (GRPO), and their combination to the Qwen2.5-Coder-1.5B model and evaluate them on GSM8K, ARC-Easy, ARC-Challenge, and AIME. Our ablations show that SFT makes GRPO training more stable, with the combined model achieving the strongest overall performance across GSM8K and ARC benchmarks. The evaluation results on ARC also show that our model avoids cross-task performance loss. Together, these findings demonstrate that targeted methods can substantially improve reasoning in compact open-source models.

1 Introduction

Large language models (LLMs) exhibit impressive linguistic fluency, yet their reliability on multi-step reasoning tasks—particularly mathematical problem solving and scientific question answering—remains uneven. Even well-trained models often produce coherent but logically invalid reasoning chains, revealing a gap between surface-level fluency and genuine deductive capability. This gap is especially pronounced in small models, which lack both the representational capacity and the training diversity of larger proprietary systems.

A central challenge lies in the training paradigm itself. **Supervised Fine-Tuning (SFT)** exposes models to curated chain-of-thought demonstrations, improving stylistic fluency but not enforcing internal logical correctness; models learn how correct reasoning *looks*, not how to construct it reliably. In contrast, reinforcement-based approaches can optimize for answer-level correctness, yet they struggle with sparse rewards and unstable updates—limitations that disproportionately affect small models, where exploratory rollouts rarely produce correct solutions. As a result, current open models under 2B parameters remain far from competitive on high-difficulty benchmarks such as GSM8K, ARC-Challenge, or AIME, despite promising early work from projects like DeepSeek-Math.

What remains unclear is how much reasoning improvement can be achieved in small models without additional large-scale pre-training or proprietary infrastructure. Prior pipelines typically combine pre-training, supervised fine-tuning, and reinforcement optimization simultaneously, making it impossible to isolate which component actually drives the observed improvements. This lack of clarity hinders reproducibility and prevents smaller labs from deploying efficient, transparent reasoning models suited for constrained environments.

To address this gap, our project systematically isolates and evaluates the contributions of **Supervised Fine-Tuning (SFT)** and **Group Relative Policy Optimization (GRPO)** on a compact open-source model: Qwen2.5-Coder-1.5B. SFT provides structured reasoning examples, while GRPO directly optimizes final-answer correctness using stable, group-normalized advantage estimates. By applying each method independently and in combination—and testing all resulting models across GSM8K,

ARC-Easy, ARC-Challenge, and AIME—we quantify how these techniques shape reasoning behavior in small models and where their limitations emerge.

From a research standpoint, this controlled design allows us to distinguish imitation-driven gains from reward-driven gains, clarifying how reasoning emerges in low-parameter regimes. Ultimately, our findings aim to establish a clearer foundation for building transparent, efficient reasoning pipelines that are accessible beyond large industrial labs.

2 Related Work

2.1 Chain-of-Thought and Reasoning Datasets

The Chain-of-Thought (CoT)[4] paradigm has emerged as a key technique to enhance multi-step reasoning in large language models (LLMs). By training or prompting models to generate explicit intermediate reasoning steps, CoT helps improve both interpretability and correctness in solving complex problems. Datasets such as GSM8K, MATH, and AIME have become standard benchmarks for evaluating mathematical reasoning capabilities. These efforts established the foundation for structured reasoning supervision, inspiring subsequent approaches that explicitly train models on reasoning traces—a cornerstone of our supervised fine-tuning (SFT) phase.

2.2 Supervised Fine-Tuning and Data Augmentation

Supervised fine-tuning on reasoning traces remains one of the most effective and reproducible approaches for enhancing reasoning performance in smaller models. The MetaMath framework [3] exemplifies this direction by augmenting mathematical datasets through paraphrasing and generating semantically equivalent question variants, significantly improving results on arithmetic tasks like GSM8K. However, SFT methods generally focus on imitation learning, lacking mechanisms to explicitly correct logical inconsistencies or optimize reasoning strategies. Consequently, their improvements plateau on high-difficulty tasks such as MATH and AIME. This limitation motivates integrating reinforcement learning (RL) to refine reasoning quality beyond imitation-based learning.

2.3 Reinforcement Learning for Reasoning

Reinforcement learning has been widely applied to align model behavior with human preferences (e.g., RLHF) and optimize specific objectives in language generation. Yet applying RL to reasoning remains challenging due to sparse reward signals and training instability. Recent studies—particularly DeepSeek-R1 [1] and DeepSeekMath [2]—proposed reward formulations that explicitly incentivize logical correctness and stepwise reasoning. Moreover, the Group Relative Policy Optimization (GRPO)[6] algorithm was introduced as an improvement over PPO to stabilize training and balance group-level performance optimization. These developments inform our project’s use of GRPO as the core RL mechanism to enhance reasoning coherence and correctness.

3 Background

Our project employs a two-phase training pipeline that combines Supervised Fine-Tuning (SFT) and Reinforcement Learning (RL) to enhance the reasoning ability of a small-scale open language model. The goal is to improve both step-by-step reasoning coherence and final-answer correctness in mathematical and coding tasks.

3.1 Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning aligns the model’s output distribution with human-annotated reasoning traces through imitation learning. Given a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \tag{1}$$

where x_i represents an input prompt (e.g., a math problem) and y_i the corresponding chain-of-thought reasoning and final answer, the fine-tuning objective minimizes the negative log-likelihood of reference outputs under the model’s conditional distribution:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} \sum_{t=1}^T \log p_{\theta}(y_t \mid y_{<t}, x). \quad (2)$$

where θ represents the model parameters, $\mathcal{D}$ denotes the dataset containing input-output pairs, T is the length of the target sequence, and $y_{<t}$ indicates the partial sequence of tokens preceding time step t . This objective encourages the model p_{θ} to replicate step-by-step reasoning consistent with human demonstrations. In practice, SFT is implemented using standard cross-entropy loss with teacher forcing over tokenized reasoning traces. The process improves logical structure and interpretability but does not explicitly penalize inconsistencies or incorrect conclusions, motivating subsequent reinforcement-based refinement.

3.2 Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization (GRPO) eliminates the need for a separate value function critic by using group-based score normalization to estimate the baseline. For each input prompt x , GRPO samples a group of G outputs $\{y_1, y_2, \dots, y_G\}$ from the current policy $\pi_{\theta_{\text{old}}}$. The optimization objective maximizes the following surrogate loss:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \epsilon \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i(\theta) A_i, \text{clip}(\rho_i(\theta), 1 - \epsilon, 1 + \epsilon) A_i) - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right] \quad (3)$$

Here, $\rho_i(\theta)$ represents the probability ratio between the current policy π_{θ} and the old policy $\pi_{\theta_{\text{old}}}$ for the i -th output:

$$\rho_i(\theta) = \frac{\pi_{\theta}(y_i \mid x)}{\pi_{\theta_{\text{old}}}(y_i \mid x)} \quad (4)$$

The key innovation of GRPO lies in the calculation of the advantage A_i . Instead of relying on a learned value function, A_i is computed by normalizing the rewards within the sampled group. Specifically, for the i -th output with reward r_i , the advantage is defined as:

$$A_i = \frac{r_i - \mu_{\text{group}}}{\sigma_{\text{group}}} \quad (5)$$

where μ_{group} and σ_{group} denote the mean and standard deviation of the rewards $\{r_1, \dots, r_G\}$ within the group, respectively. The term $\mathbb{D}_{\text{KL}}$ serves as a penalty to prevent the model from deviating excessively from the reference policy π_{ref} (typically the SFT model).

4 Methods

This section describes the training procedures used to evaluate how Supervised Fine-Tuning (SFT) and Group Relative Policy Optimization (GRPO) independently and jointly influence reasoning performance in a small model. Our goal is to isolate these stages without confounding effects from additional pre-training or architectural changes. We therefore begin with a fixed 1.5B-parameter backbone, apply SFT and GRPO under controlled and comparable conditions, and treat each training strategy as an experimental variable. The following subsections detail the base model, the implementation of SFT, GRPO, and their combination, as well as the rationale for purposely excluding any further pre-training.

4.1 Base Model

All experiments utilize the **Qwen2.5-Coder-1.5B** model (8), an open-weight, 1.5B-parameter code-centric LLM. This model is particularly well-suited for our setting because it is compact enough to facilitate computationally efficient reinforcement learning while maintaining non-trivial reasoning capabilities.

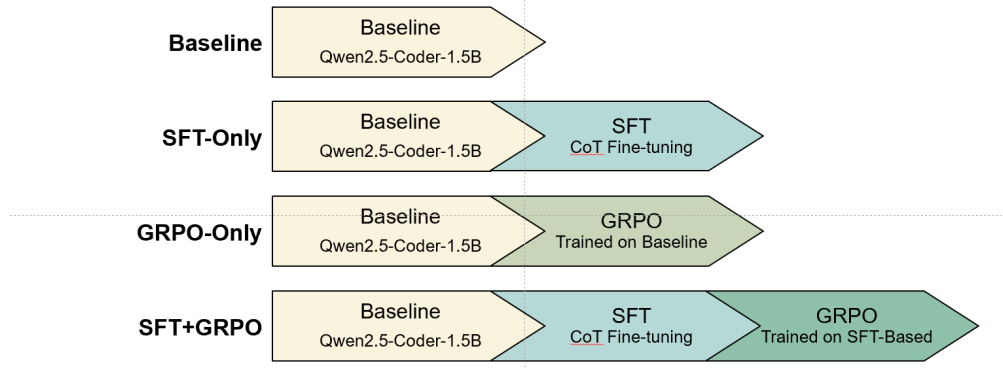


Figure 1: **Overview of Experimental Configurations.** We compare four primary setups: the specific Baseline, SFT-only, GRPO-only, and the combined SFT+GRPO pipeline. The logical flow illustrates how initialization strategies differ across methods.

4.2 Applying SFT

We fine-tuned the **Qwen2.5-1.5B-SFT** model using the *LLaMA-Factory* framework under the supervised fine-tuning (SFT) stage. Parameter-efficient tuning was conducted via LoRA with rank 8, $\alpha = 16$, and dropout 0. Training used the AdamW optimizer with an initial learning rate of 5×10^{-5} and a cosine decay schedule for 3 epochs. The maximum sequence length was 2048, and the effective batch size was 16 (batch size 2 with 8 gradient accumulation steps). Gradient clipping was set to 1.0, mixed precision training used bf16, and 10% of data was reserved for validation.

This yields the **SFT-only** model, which learns to imitate human CoT traces but does not use reward signals.

4.3 Applying GRPO

The **Qwen2.5-1.5B-GRPO** model was quantized to 4-bit NF4 and fine-tuned with LoRA ($r = 16$, $\alpha = 32$, dropout 0.05) applied to attention and MLP layers. Training used an effective batch size of 4 per GPU for 1 epoch, with learning rate 5×10^{-5} , warmup ratio 0.1, and gradient clipping at 1.0. Each prompt generated 4 candidate completions (temperature=0.9, top-p=0.9, top-k=50), and rewards were assigned based on the correctness of the final numeric answer with KL penalty $\beta = 0.01$.

4.4 Applying Combined SFT+GRPO

To study complementarity, we also apply GRPO starting from the SFT-tuned policy. That is, the SFT model serves as the reference model and initialization for the GRPO phase, while the same reward definition and hyperparameters as in §4.2 and §4.3 are used.

4.5 Experimental Design Rationale

We exclude any additional pre-training to cleanly isolate the effects of **SFT** and **GRPO** on reasoning performance. In typical large-scale reasoning pipelines, pre-training, supervised adaptation, and reinforcement learning are applied sequentially, with their contributions intertwined. This coupling makes it difficult to determine whether observed improvements arise from broad pre-training, from structured supervision, or from reward-guided optimization. By holding the backbone fixed and varying only SFT and GRPO, our setup treats each method as a controlled intervention within an ablation-style framework, allowing for clear causal attribution.

This design yields four directly comparable models—Baseline, SFT-only, GRPO-only, and SFT+GRPO—that differ exclusively in their optimization procedures. Because all other factors are held constant, performance differences can be attributed specifically to the training strategies under investigation rather than to pre-training scale or data diversity. The following section evaluates

these models across multiple reasoning benchmarks to assess how each optimization method shapes reasoning behavior in compact language models.

5 Experiments

5.1 Training Setup

We evaluate four models produced by the training procedures described in Section 4:

- (1) The Baseline model,
- (2) SFT-only,
- (3) GRPO-only, and
- (4) SFT+GRPO, where GRPO is initialized from the SFT model.

All models are trained on the same GSM8K training split to ensure comparable exposure to supervision. No additional pre-training or external datasets are introduced, enabling a direct comparison of how each optimization stage shapes reasoning performance. After training, all models are evaluated under identical decoding and prompting conditions across multiple benchmarks.

5.2 Evaluation Datasets

We assess mathematical and logical reasoning using four benchmarks with varying levels of difficulty and distributional shift:

- **GSM8K** (9) — Grade-school arithmetic problems with short chain-of-thought solutions. This dataset serves as our *only* source for training.
- **ARC-Easy** and **ARC-Challenge** (10) — Subsets of the AI2 Reasoning Challenge targeting commonsense and multi-step scientific reasoning, formatted as multiple-choice questions.
- **AIME** — Competition-level algebraic and geometric problems requiring complex symbolic manipulation. We utilize the 2024 (11) and 2025 (12) problem sets to assess frontier reasoning capabilities.

GSM8K serves as our primary data source; specifically, we train exclusively on its training split. The held-out GSM8K test split, along with ARC and AIME, is used strictly for evaluation to measure both in-domain performance and out-of-distribution generalization

5.3 Prompting and Evaluation Protocol

Because each benchmark differs in structure and formatting, we employ dataset-specific prompting strategies while keeping evaluation metrics consistent across models.

GSM8K

We use a few-shot chain-of-thought prompt containing solved examples in natural language. Models are instructed to “think step by step” and produce a final numeric answer. Performance is measured using exact-match accuracy.

ARC-Easy / ARC-Challenge

Few-shot prompts include several example multiple-choice questions paired with correct labels. Models must output only a single letter corresponding to the chosen option. Accuracy is computed as the fraction of exact letter matches.

AIME

Preliminary experimentation revealed that the models struggled to interpret LaTeX notation. To mitigate this, we design prompts containing LaTeX-formatted question-answer examples and instruct the model to output only the final integer. Evaluation again uses exact-match accuracy. Across all

benchmarks, models are evaluated using a consistent decoding configuration, ensuring that differences in performance reflect training effects rather than prompting or sampling artifacts.

5.4 Summary

This experimental setup provides a controlled framework for comparing the reasoning behavior of four models that differ only in their optimization strategies. By combining standardized evaluation with benchmark-specific prompting, we isolate the contributions of SFT, GRPO, and their interaction, enabling a clear understanding of how each method affects reasoning performance in compact language models. The next section reports quantitative results across all datasets.

6 Results

Table 1: Accuracy (%) of Different Training Strategies on GSM8K and ARC.

Dataset	Method	Accuracy (%)
GSM8K	Baseline	56.61
	Only SFT	61.17
	Only GRPO	67.40
	SFT + GRPO	68.47
ARC Easy	Baseline	73.20
	Only SFT	74.70
	Only GRPO	76.20
	SFT + GRPO	76.00
ARC Challenge	Baseline	57.70
	Only SFT	57.90
	Only GRPO	57.50
	SFT + GRPO	60.60

Our two-stage SFT+GRPO approach consistently improves both mathematical reasoning and cross-task performance. On ARC-easy, performance is fully preserved and even slightly improved, indicating no forgetting and positive transfer to simpler reasoning tasks. In contrast, ARC-challenge reveals a more nuanced behavior: GRPO alone exhibits a small performance drop, suggesting mild catastrophic forgetting when reward-based optimization is applied without a stable initialization. However, combining SFT with GRPO not only prevents degradation but also yields the strongest gains across all tasks. Overall, these results demonstrate that SFT+GRPO provides the most stable and generalizable training strategy, enhancing in-domain mathematical reasoning while avoiding cross-task performance loss and achieving improved robustness on challenging out-of-distribution reasoning tasks.

SFT makes GRPO training process more stable. With SFT + GRPO, the KL divergence remains consistently within the 0.02–0.05 range, with only a brief spike around step 600 that quickly self-corrects. This behavior indicates that policy updates are well-regulated by the KL penalty, and the model stays close to the SFT-initialized distribution throughout training. In contrast, GRPO-only exhibits severe KL divergence: after a short initial period near zero, the KL value rapidly increases and repeatedly spikes to 10–30, with no signs of stabilization. The large oscillations indicate drastic and uncontrolled shifts in the policy distribution. Combined with the observed increase in gradient norm and policy entropy, this behavior reflects a clear case of policy collapse and optimization instability. These results demonstrate that SFT initialization is essential for stable GRPO training. It constrains the policy within a safe region of the distribution space, whereas training GRPO directly from the base model leads to uncontrolled divergence and failure to converge.



(a) **GRPO-only**. Initialized directly from the base model.



(b) **SFT+GRPO**. Initialized from the SFT checkpoint.

Figure 2: Impact of SFT initialization on GRPO training stability. These large-scale training dynamics plots illustrate the stabilizing effect of supervised warm-up. Subfigure (a) shows the "cold start" scenario (from base model), characterized by high initial volatility and a sharp spike in KL divergence as the policy rapidly shifts. Subfigure (b) demonstrates the "warm start" scenario (from SFT checkpoint), showing significantly smoother accuracy improvements, stable gradient norms, and consistently low KL divergence, indicating that SFT effectively prevents early-stage training instabilities.

7 Discussion

7.1 Effect on GSM8K

The GSM8K results show a clear and interpretable pattern: SFT and GRPO improve performance through different mechanisms, and combining them yields the strongest overall gains. SFT alone increases accuracy from 56.61% to 61.17%, confirming that explicit chain-of-thought demonstrations help the model internalize structured multi-step reasoning templates. However, SFT’s improvements plateau quickly, and the model shows tendency toward rigid, template-like solutions—an expected outcome when optimization focuses solely on imitation.

GRPO drives a substantially larger improvement, raising accuracy to 67.40% despite not receiving any gold reasoning traces. This indicates that directly optimizing for answer-level correctness provides a stronger learning signal than imitation alone. GRPO encourages the policy to explore alternative reasoning paths and discard trajectories that systematically fail, producing more reliable final answers even when intermediate reasoning is imperfect.

The combined **SFT+GRPO** model achieves the highest accuracy at 68.47%. This result is significant: beginning GRPO from an SFT-initialized policy produces more stable and productive exploration than starting from the raw base model. In other words, SFT equips the model with an initial scaffold of structured reasoning, and GRPO fine-tunes that scaffold toward reward-aligned decision-making. The interaction is genuinely complementary rather than redundant, and the performance gains on GSM8K reflect this synergy.

We conducted an additional experiment to examine whether the benefits of the two-stage SFT+GRPO model depend on explicitly using the SFT component at inference time. Specifically, we evaluated a variant trained with SFT+GRPO but hid the SFT module during testing, using only the GRPO-updated policy for GSM8K inference. The model achieved an accuracy of 68.24% on the GSM8K test set, closely matching the 68.47% accuracy of the full SFT+GRPO model. This indicates that SFT mainly serves as a training-time prior that stabilizes optimization and prevents catastrophic forgetting, while its explicit involvement is no longer needed after GRPO refinement. Thus, the GRPO-trained policy effectively internalizes the advantages conferred by SFT.

7.2 Generalization to ARC reasoning datasets

The ARC results further clarify the relationship between SFT, GRPO, and the underlying reasoning demands of each dataset. On **ARC-Easy**, the baseline already performs well at 73.20%, suggesting that many questions rely on shallow commonsense reasoning rather than complex computation. In this regime, GRPO alone yields the best accuracy (76.20%), outperforming both SFT and SFT+GRPO. This pattern is consistent with ARC-Easy’s task structure: because the questions are multiple-choice and require less explicit step-by-step reasoning, answer-level optimization directly improves the model’s decision boundary without being constrained by SFT’s preference for verbose, math-oriented explanations.

The situation changes on **ARC-Challenge**, where reasoning complexity is significantly higher. Here, neither SFT nor GRPO alone provides meaningful improvement over the baseline. The modest gains from both methods individually indicate that the tasks require deeper compositional reasoning than what either imitation-based tuning or reward-only optimization can supply by itself. In contrast, **SFT+GRPO** achieves a clear improvement to 60.60%, showing that the two-stage pipeline is more capable of handling difficult, compositional reasoning tasks. The SFT stage supplies structural priors that stabilize GRPO’s updates in a domain where rewards are sparse and exploratory rollouts are unlikely to stumble onto correct solutions. GRPO then adjusts these priors toward answer-level correctness, producing a model that is both structured and reward-aligned.

These results reinforce the conclusion that SFT and GRPO function as complementary mechanisms. Their relative contributions depend on the dataset’s reasoning demands: GRPO dominates when answers can be reached through shallow decision-making, while the combination is essential when the task requires multi-step, compositional reasoning under sparse reward signals.

7.3 Failure on AIME and Limits of the 1.5B Model

Across all four models, accuracy on AIME is effectively zero, and this failure is systematic rather than incidental. The primary cause is the severe distribution mismatch between GSM8K—the sole source of SFT and GRPO training data—and the mathematical structure of AIME. GSM8K problems involve shallow arithmetic reasoning with short natural-language explanations, while AIME requires multi-step symbolic manipulation, algebraic transformations, and geometric insight. Because none of these competencies appear in the training corpus, the model consistently falls back on GSM8K-style heuristics that are inappropriate for contest mathematics.

Formatting and representational limitations compound the problem. AIME problems rely heavily on LaTeX notation; even with LaTeX-aware few-shot prompts, the 1.5B model frequently mis-parses expressions or applies invalid transformations. This behavior is consistent with its pretraining profile: Qwen2.5-Coder-1.5B has limited exposure to high-entropy mathematical syntax, and its tokenizer and embedding distribution are not optimized for symbolic reasoning. As a result, the model often fails before any meaningful reasoning begins.

Model capacity further restricts performance. At 1.5B parameters, the model lacks the representational depth required to store and execute the structured algorithms needed for AIME-level problems. The reinforcement-learning setup offers no remedy, because the GRPO reward is a sparse, final-answer signal. For AIME, where correct answers are extremely difficult to obtain by random exploration, GRPO provides essentially no gradient information. The outcome—uniformly near-zero accuracy—confirms that the pipeline cannot extrapolate from GSM8K to competition mathematics under its current design.

7.4 Implications and Future Directions

The AIME results demonstrate that improving mid-level reasoning on GSM8K and ARC does not translate to high-level symbolic reasoning. The pipeline fails because the model never encounters the mathematical structures required for transfer. Introducing datasets such as MATH or the AIME archives into SFT would directly address this gap by exposing the model to the algebraic patterns, geometric arguments, and LaTeX structures that are absent from GSM8K. Without such data, the model’s internal reasoning space remains fundamentally misaligned with the target domain.

The reward design in GRPO also limits progress. A binary final-answer reward is insufficient for tasks where valid trajectories are exceedingly rare. In AIME-style problems, the probability of randomly producing a correct numeric answer is effectively zero, so GRPO receives no actionable signal. Dense, step-level rewards—via algebraic consistency checks, verifier models, or symbolic tool integration—would provide the gradient information needed to shape long reasoning chains rather than merely their endpoints.

Finally, the failure raises a capacity question. A 1.5B model may simply be too small to represent the algorithmic and symbolic machinery required for competition mathematics. Larger Qwen models or architectures pretrained on math-heavy corpora routinely outperform small models on formal reasoning tasks, suggesting that the limitation is structural, not incidental. Replicating our SFT+GRPO pipeline on larger backbones would clarify whether the bottleneck lies in data and reward design, or whether small models are intrinsically incapable of mastering high-level symbolic reasoning.

8 Conclusion

Our results show that SFT and GRPO each improve small-scale LLM reasoning, but their combination consistently delivers the strongest and most stable gains across diverse benchmarks. While this pipeline effectively enhances grade school level mathematical and commonsense reasoning, it fails to generalize to competition-level tasks such as AIME, where symbolic complexity, formatting, and capacity demands exceed what a 1.5B model can handle. These findings underscore both the promise and the limitations of lightweight alignment methods for reasoning. Future work will require richer mathematical training data, denser reward signals, and larger or more specialized base models to achieve robust high-level reasoning performance.

References

- [1] Guo, D., Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Li, Y., Wu, Y., & Guo, D. (2025) DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*. Available at: <https://arxiv.org/abs/2501.12948>.
- [2] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, M., Li, Y., Wu, Y., & Guo, D. (2024) DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*. Available at: <https://arxiv.org/abs/2402.03300>.
- [3] Yu, Z., Liu, J., Zhang, T., Zhao, H., & Zhao, H. (2023) MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. *arXiv preprint arXiv:2309.12284*. Available at: <https://arxiv.org/abs/2309.12284>.
- [4] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022) Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903*. Available at: <https://arxiv.org/abs/2201.11903>.
- [5] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaplan, J., Power, A., Skjonsberg, L., Sutskever, I., & Amodei, D. (2021) Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*. Available at: <https://arxiv.org/abs/2110.14168>.
- [6] Hugging Face, *GRPO Trainer*. https://huggingface.co/docs/trl/main/en/grpo_trainer, accessed Sept. 28, 2025.
- [7] Alibaba Qwen. (2024). *Qwen2.5-Coder-1.5B*. Hugging Face. Available at: <https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B>
- [8] Qwen Team. (2024). *Qwen2.5-Coder-1.5B*. Hugging Face. Available at: <https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B>
- [9] Cobbe, K., et al. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv preprint arXiv:2110.14168. Available at: <https://arxiv.org/abs/2110.14168>
- [10] Clark, P., et al. (2018). *Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge*. arXiv preprint arXiv:1803.05457. Available at: <https://arxiv.org/abs/1803.05457>
- [11] Maxwell-Jia. (2024). *AIME 2024 Dataset*. Hugging Face. Available at: https://huggingface.co/datasets/Maxwell-Jia/AIME_2024
- [12] Zhang, Y., & Math-AI Team. (2025). *American Invitational Mathematics Examination (AIME) 2025*. Hugging Face. Available at: <https://huggingface.co/datasets/math-ai/aime25>